M18 # Event-Driven Agents

Méthodologie Advanced

Déclencher des agents automatiquement depuis GitHub, Linear, Jira

Du pull au push

L'usage interactif de Claude Code est pull-based : vous ouvrez un terminal, vous tapez une instruction, vous itérez. L'automatisation event-driven retire l'humain de l'étape de déclenchement. Une carte passe en « In Progress » dans Linear, un agent démarre. Une issue GitHub reçoit le label `claude-fix`, un agent ouvre une branche en quelques secondes.

L'humain reste dans la boucle pour la validation finale (review de PR, approbation de merge), mais l'initiation du travail passe par le workflow de gestion de projet existant.

Architecture du pipeline

```
Source d'événement
  → Filtre (label, mention, état)
    → Extraction du contexte
      → Sélection de l'agent
        → Claude Code
          → Routage du résultat (PR, commentaire, statut)
```

Chaque composant est indépendant et remplaçable. Le filtre détermine quels événements méritent un agent ; sans filtrage strict, une rafale d'événements peut instancier des dizaines d'agents simultanément.

Sources d'événements compatibles

Source	Déclencheurs	Usage type
GitHub Issues	Création, label ajouté	Triage, fix automatique
GitHub PR	Ouverture, commentaire	Review, corrections
Linear	Changement d'état, label	Implémentation de feature
Jira	Transition, assignation	Travail technique, dette
Slack	Message, réaction emoji	Investigations rapides
PagerDuty	Incident créé	Scripts de diagnostic

Filtre d'activation

Ne pas déclencher un agent pour chaque événement. Un filtre simple en bash :

```bash
# Déclencher uniquement si label "claude-auto" présent
if [[ "$CARD_LABELS" != *"claude-auto"* ]]; then
  echo "Skipping: no claude-auto label"
  exit 0
fi
```

Pour GitHub, le trigger `@claude review` dans un commentaire de PR est le pattern le plus documenté : il déclenchement on-demand sans polluer le workflow automatique.

Exemple concret : Linear agent loop

Le pattern le plus documenté (Damian Galarza, février 2026) : Linear comme source de vérité unique, Claude Code comme implémenteur.

La description du ticket sert de prompt. Un ticket avec des critères d'acceptation clairs produit du code de qualité ; un ticket vague produit du code vague, exactement comme avec un développeur humain.

```bash
spawn_agent() {
  set -o pipefail
  claude --print --dangerously-skip-permissions \
    "Implement Linear card:
    Title: $title
    Description: $description
    1. Create branch feat/$issue_id
    2. Implement feature, run tests
    3. Open PR" 2>&1 | tee "/tmp/agent-$issue_id.log"
}
```

Garde-fous obligatoires

Idempotence : vérifier qu'une branche n'existe pas déjà avant de démarrer, pour éviter le double traitement en cas de webhook dupliqué.

Limite de concurrence : fixer un plafond selon les ressources mesurées, les limites de débit et le budget. L'augmenter seulement si les tâches terminées indépendamment justifient cette coordination.

Circuit breaker : si un agent échoue plus de 3 fois sur le même ticket, stopper l'automatisation et alerter un humain.

Revue humaine : les agents créent des PRs, les humains les approuvent. La merge vers main ne doit jamais être automatisée sans validation.

Authentification minimale

Le token Claude Code utilisé en CI doit être scoped au minimum nécessaire. Un agent de review de PR n'a pas besoin d'accès en écriture au registre npm ou aux secrets de déploiement. Séparer les tokens par type d'agent et auditer régulièrement leurs permissions effectives.